Assignment – 12.4

Name : R.Nikhil Kumar
Batch : 44
Id : 2303A52260

Task 1 :
Bubble Sort for Ranking Exam Scores

Scenario :

You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Code :

```python
# Task 1: Bubble Sort for Ranking Exam Scores
#Implement Bubble Sort in Python to sort a list of student scores.
# Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes
# Identify early-termination conditions when the list becomes sorted
# Provide a brief time complexity analysis
def bubble_sort(scores):
    n = len(scores)
    # Flag to check if any swapping occurs
    swapped = True

    # Outer loop to traverse through all elements
    for i in range(n):
        # If no swapping occurs, the list is already sorted
        if not swapped:
            break

        # Reset the swapped flag for this pass
        swapped = False

        # Inner loop to compare adjacent elements
        for j in range(0, n-i-1):
            # Compare the current element with the next element
            if scores[j] > scores[j+1]:
                # Swap if the current element is greater than the next element
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True  # Set the flag to True if a swap occurred

    return scores
# Example usage
scores = [88, 75, 92, 85, 79]
sorted_scores = bubble_sort(scores)
print("Sorted Scores:", sorted_scores)
# Time Complexity Analysis:
# The time complexity of Bubble Sort is O(n^2) in the worst and average cases, where n is the number of elements in the list. This is because in the worst case, each element needs to be compared with every other element.
# However, if the list is already sorted or nearly sorted, the time complexity can be O(n) due to the early-termination condition when no swaps are made.
```

Output :

```
Sorted Scores: [75, 79, 85, 88, 92]
```

Task 2 :
Improving Sorting for Nearly Sorted Attendance Records

Scenario :

You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates

Code :

```python
# Task 2: Improving Sorting for Nearly Sorted Attendance Records
#Start with a Bubble Sort implementation.
#Review the problem and suggest a more suitable sorting algorithm
#Generate an Insertion Sort implementation
#Explain why Insertion Sort performs better on nearly sorted data
#Compare execution behavior on nearly sorted input
def bubble_sort(scores):
    n = len(scores)
    swapped = True

    for i in range(n):
        if not swapped:
            break
        swapped = False

        for j in range(0, n-i-1):
            if scores[j] > scores[j+1]:
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True

    return scores
# Insertion Sort Implementation
def insertion_sort(scores):
    n = len(scores)

    # Traverse through 1 to n
    for i in range(1, n):
        key = scores[i]
        j = i - 1

        # Move elements of scores[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and scores[j] > key:
            scores[j + 1] = scores[j]
            j -= 1
        scores[j + 1] = key

    return scores
# Example usage for comparison
nearly_sorted_scores = [88, 75, 92, 85, 79]  # This list is nearly sorted
# Sort using Bubble Sort
sorted_scores_bubble = bubble_sort(nearly_sorted_scores.copy())
print("Sorted Scores with Bubble Sort:", sorted_scores_bubble)
# Sort using Insertion Sort
sorted_scores_insertion = insertion_sort(nearly_sorted_scores.copy())
print("Sorted Scores with Insertion Sort:", sorted_scores_insertion)
# Explanation of Efficiency Differences:
# Insertion Sort performs better on nearly sorted data because it minimizes the number of comparisons and movements
# needed to sort the list. When the list is nearly sorted,
# Insertion Sort can quickly place elements in their correct positions with fewer shifts,resulting in a time complexity of O(n) in the best case.
# In contrast, Bubble Sort will still require O(n^2) time in the worst case,
# as it will need to compare and potentially swap each element multiple times, regardless of the initial order.
```

Ouput :

```
Sorted Scores with Bubble Sort: [75, 79, 85, 88, 92]
Sorted Scores with Insertion Sort: [75, 79, 85, 88, 92]
```

Task 3 :

Searching Student Records in a Database

Scenario :

You are developing a student information portal where users search for student records by roll number.

Code :

```python
# Task 3: Searching Student Records in a Database
# Implement:
# Linear Search for unsorted student data
# Binary Search for sorted student data
# Add docstrings explaining parameters and return values
# Explain when Binary Search is applicable
# Highlight performance differences between the two searches
# Linear Search Implementation
def linear_search(student_records, roll_number):
    """
    Perform a linear search to find a student record by roll number.
    Parameters:
    student_records (list): A list of student records, where each record is a dictionary containing 'roll_number' and other details. roll_number (int): The roll number to search for.
    Returns:
    dict: The student record if found, otherwise None.
    """
    for record in student_records:
        if record['roll_number'] == roll_number:
            return record
    return None
# Binary Search Implementation
def binary_search(student_records, roll_number):
    """
    Perform a binary search to find a student record by roll number.
    Parameters:
    student_records (list): A sorted list of student records, where each record is a dictionary containing 'roll_number' and other details. roll_number (int): The roll number to search for.
    Returns:dict: The student record if found, otherwise None.
    Note:
    Binary Search is applicable only when the list of student records is sorted by roll number. It works by repeatedly dividing the search interval in half, which allows it
    to achieve a time complexity of O(log n) in the best and average cases.
    In contrast, Linear Search has a time complexity of O(n) in the worst case, as it may need to check each record until it finds the target or reaches the end of the list.
    """
    left, right = 0, len(student_records) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if student_records[mid]['roll_number'] == roll_number:
            return student_records[mid]
        elif student_records[mid]['roll_number'] < roll_number:
            left = mid + 1
        else:
            right = mid - 1
    return None
# Example usage
student_records = [
    {'roll_number': 101, 'name': 'Alice'},
    {'roll_number': 102, 'name': 'Bob'},
    {'roll_number': 103, 'name': 'Charlie'},
    {'roll_number': 104, 'name': 'David'},
    {'roll_number': 105, 'name': 'Eve'}
]
# Searching for a student record using Linear Search
result_linear = linear_search(student_records, 103)
print("Linear Search Result:", result_linear)
# Searching for a student record using Binary Search (requires sorted list)
sorted_student_records = sorted(student_records, key=lambda x: x['roll_number'])
result_binary = binary_search(sorted_student_records, 103)
print("Binary Search Result:", result_binary)
# Performance Comparison:
# Linear Search will check each record one by one, resulting in a time complexity of O(n) in the worst case, especially if the target record is at the end of the list or not present at all.
# Binary Search, on the other hand, will quickly narrow down the search space by halving
#  it with each iteration, achieving a time complexity of O(log n) in the best and average cases. However, it requires the list to be sorted beforehand, which may add additional overhead
#  if the data is not already sorted. In practice, for small datasets, the difference in performance may not be significant, but for larger datasets, Binary Search can provide
#  a substantial improvement in search efficiency compared to Linear Search.
```

Output :

```
Linear Search Result: {'roll_number': 103, 'name': 'Charlie'}
Binary Search Result: {'roll_number': 103, 'name': 'Charlie'}
```

Task 4 :

Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario :

You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Code :

```python
# Task 4: Choosing Between Quick Sort and
# Merge Sort for Data Processing
# Provide AI with partially written recursive functions for:
# Quick Sort
# Merge Sort
# Ask AI to:
# Complete the recursive logic
# Add meaningful docstrings
# Explain how recursion works in each algorithm
# Test both algorithms on:
# Random data
# Sorted data
# Reverse-sorted data
# Quick Sort Implementation
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.
    Parameters:arr (list): A list of elements to be sorted.
    Returns:list: A new list containing the sorted elements.
    Explanation:Quick Sort is a divide-and-conquer algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays,
    according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. This process continues until the base case of an empty array or
    a single element is reached, which is inherently sorted.
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[len(arr) // 2]  # Choosing the middle element as the pivot
        left = [x for x in arr if x < pivot]  # Elements less than the pivot
        middle = [x for x in arr if x == pivot]  # Elements equal to the pivot
        right = [x for x in arr if x > pivot]  # Elements greater than the pivot
        return quick_sort(left) + middle + quick_sort(right)  # Recursively sort and combine
# Merge Sort Implementation
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.
    Parameters:arr (list): A list of elements to be sorted.
    Returns:list: A new list containing the sorted elements.
    Explanation:Merge Sort is a divide-and-conquer algorithm that works by recursively splitting the array into halves until it reaches arrays of size one or zero,
      which are inherently sorted.
    Then, it merges those smaller arrays back together in a sorted manner.
      The merging process involves comparing the elements of the two halves and combining them in order until all elements are merged back into a single sorted array.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2  # Finding the mid of the array
    left_half = merge_sort(arr[:mid])  # Recursively sorting the left half
    right_half = merge_sort(arr[mid:])  # Recursively sorting the right half
    return merge(left_half, right_half)  # Merging the sorted halves
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
    Parameters:left (list): A sorted list of elements. right (list): Another sorted list of elements.
    Returns:list: A new list containing all elements from both input lists in sorted order.
    """
    result = []
    i = j = 0
    # Merge the two lists while maintaining sorted order
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    # If there are remaining elements in the left list, add them to the result
    result.extend(left[i:])
    # If there are remaining elements in the right list, add them to the result
    result.extend(right[j:])
    return result
# Testing both algorithms on different types of data
random_data = [5, 2, 9, 1, 5, 6]
sorted_data = [1, 2, 3, 4, 5, 6]
reverse_sorted_data = [6, 5, 4, 3, 2, 1]
# Testing Quick Sort
print("Quick Sort Results:")
print("Random Data:", quick_sort(random_data.copy()))
print("Sorted Data:", quick_sort(sorted_data.copy()))
print("Reverse Sorted Data:", quick_sort(reverse_sorted_data.copy()))
# Testing Merge Sort
print("\nMerge Sort Results:")
print("Random Data:", merge_sort(random_data.copy()))
print("Sorted Data:", merge_sort(sorted_data.copy()))
print("Reverse Sorted Data:", merge_sort(reverse_sorted_data.copy()))
# Performance Comparison:
# Quick Sort:
# Best Case: O(n log n) - occurs when the pivot divides the array into two equal halves.
# Average Case: O(n log n) - occurs on average for random pivots.
# Worst Case: O(n^2) - occurs when the smallest or largest element is always chosen as the pivot (e.g., already sorted or reverse sorted data).
# Merge Sort:
# Best Case: O(n log n) - occurs regardless of the initial order of the data.
# Average Case: O(n log n) - occurs regardless of the initial order of the data
# Worst Case: O(n log n) - occurs regardless of the initial order of the data.
# In summary, Merge Sort has a consistent time complexity of O(n log n) across all
# cases, while Quick Sort can degrade to O(n^2) in the worst case, making Merge Sort a more reliable choice for sorting large datasets,
#  especially when the data may be nearly sorted or reverse sorted.
```

Output :

```
Quick Sort Results:
Random Data: [1, 2, 5, 5, 6, 9]
Sorted Data: [1, 2, 3, 4, 5, 6]
Reverse Sorted Data: [1, 2, 3, 4, 5, 6]

Merge Sort Results:
Random Data: [1, 2, 5, 5, 6, 9]
Sorted Data: [1, 2, 3, 4, 5, 6]
Reverse Sorted Data: [1, 2, 3, 4, 5, 6]
```

Task 5 :

Optimizing a Duplicate Detection Algorithm

Scenario :

You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Code :

```python
# Task 5: Optimizing a Duplicate Detection Algorithm
# Write a naive duplicate detection algorithm using nested loops.
# Analyze the time complexity
# Suggest an optimized approach using sets or dictionaries
# Rewrite the algorithm with improved efficiency
# Compare execution behavior conceptually for large input sizes
def naive_duplicate_detection(user_ids):
    """
    Detects duplicate user IDs in a list using a brute-force approach.
    Parameters:user_ids (list): A list of user IDs to be checked for duplicates.
    Returns:list: A list of duplicate user IDs found in the input list.
    Time Complexity: O(n^2) - This is because each user ID is compared with every other user ID in the list, leading to n*(n-1)/2 comparisons in the worst case.
    """
    duplicates = []
    n = len(user_ids)

    for i in range(n):
        for j in range(i + 1, n):
            if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
                duplicates.append(user_ids[i])

    return duplicates
# Optimized Duplicate Detection Algorithm using Sets
def optimized_duplicate_detection(user_ids):
    """
    Detects duplicate user IDs in a list using an optimized approach with sets.
    Parameters:user_ids (list): A list of user IDs to be checked for duplicates.
    Returns:list: A list of duplicate user IDs found in the input list.
    Time Complexity: O(n) - This is because each user ID is processed once to check for duplicates using a set, which provides O(1) average time complexity for membership checks.
    Explanation: The optimized algorithm uses a set to keep track of seen user IDs. As we iterate through the list, we check if the current user ID is already in the set.
      If it is, it's a duplicate and we add it to the duplicates list. If not, we add it to the set. This way, we avoid the need for nested loops and significantly reduce
      the time complexity from O(n^2) to O(n).
    """
    seen = set()
    duplicates = []

    for user_id in user_ids:
        if user_id in seen and user_id not in duplicates:
            duplicates.append(user_id)
        else:
            seen.add(user_id)

    return duplicates
# Example usage
user_ids = ['user1', 'user2', 'user3', 'user1', 'user4', 'user2']
print("Naive Duplicate Detection:", naive_duplicate_detection(user_ids))
print("Optimized Duplicate Detection:", optimized_duplicate_detection(user_ids))
# Performance Comparison:
# The naive duplicate detection algorithm has a time complexity of O(n^2) because it uses
# nested loops to compare each user ID with every other user ID. This means that as the input size grows, the number of comparisons increases quadratically,
# leading to significant performance degradation for large datasets.
# In contrast, the optimized duplicate detection algorithm has a time complexity of O(n) because it uses a set to track seen user IDs.
#  Each user ID is processed once, and membership checks in a set are O(1) on average. This means that even as the input size grows, the number of operations increases linearly,
#  making it much more efficient for large datasets. In practice, the optimized algorithm will perform significantly better than the naive approach, especially as the
# number of user IDs increases.
```

Output :

```
Naive Duplicate Detection: ['user1', 'user2']
Optimized Duplicate Detection: ['user1', 'user2']
```